All-Pairs Shortest Path Algorithms

Design and Analysis of Algorithms

Brian C. Dean

School of Computing

Clemson University

The Floyd-Warshall Algorithm

- Let c(i, j) be the cost of an edge from node i to node j.
  - If there is no edge, set c(i, j) = ∞.
  - Also, set c(i, i) = 0.
- After running the remarkably simple $O(n^3)$ Floyd-Warshall algorithm…

```
For k = 1 to n
  For i = 1 to n
    For j = 1 to n
      If c[i,k] + c[k,j] < c[i,j], c[i,j] = c[i,k] + c[k,j]
```

- … c(i, j) now gives us the cost of a shortest path from i to j!
- So we've transformed c from an n x n matrix of edge costs to an n x n matrix of shortest path costs.

The Floyd-Warshall Algorithm

```
For k = 1 to n
  For i = 1 to n
    For j = 1 to n
      If c[i,k] + c[k,j] < c[i,j], c[i,j] = c[i,k] + c[k,j]
```

- Works even if there are negative-cost edges (can detect negative cycles if c[i, i] ever becomes negative).
- Correctness follows from analysis as a DP algorithm:
  - Let $c^k[i, j]$ denote the cost of a shortest $i \rightarrow j$ path that uses only nodes 1 … k as potential intermediate nodes along the path.
  - Initially, $c^0[i, j]$ = the cost of edge (i, j).
  - Moreover, $c^n[i, j]$ = the cost of a shortest $i \rightarrow j$ path.
  - DP recursion: $c^k[i, j] = \min\{ c^{k-1}[i, j],\ c^{k-1}[i, k] + c^{k-1}[k, j] \}$

The Floyd-Warshall Algorithm

```
For k = 1 to n
  For i = 1 to n
    For j = 1 to n
      c^k[i, j] = min{c^(k-1)[i, j], c^(k-1)[i, k] + c^(k-1)[k, j]}
```

- Works even if there are negative-cost edges (can detect negative cycles if c[i, i] ever becomes negative).
- Correctness follows from analysis as a DP algorithm:
  - Let $c^k[i, j]$ denote the cost of a shortest $i \rightarrow j$ path that uses only nodes 1 … k as potential intermediate nodes along the path.
  - Initially, $c^0[i, j]$ = the cost of edge (i, j).
  - Moreover, $c^n[i, j]$ = the cost of a shortest $i \rightarrow j$ path.
  - DP recursion: $c^k[i, j] = \min\{ c^{k-1}[i, j],\ c^{k-1}[i, k] + c^{k-1}[k, j] \}$

The Floyd-Warshall Algorithm

```
For k = 1 to n
  For i = 1 to n                          c^k[i, k]     c^k[k, j]
    For j = 1 to n                          ||            ||
      c^k[i, j] = min{c^(k-1)[i, j], c^(k-1)[i, k] + c^(k-1)[k, j]}
```

- Works even if there are negative-cost edges (can detect negative cycles if c[i, i] ever becomes negative).
- Correctness follows from analysis as a DP algorithm:
  - Let $c^k[i, j]$ denote the cost of a shortest $i \rightarrow j$ path that uses only nodes 1 … k as potential intermediate nodes along the path.
  - Initially, $c^0[i, j]$ = the cost of edge (i, j).
  - Moreover, $c^n[i, j]$ = the cost of a shortest $i \rightarrow j$ path.
  - DP recursion: $c^k[i, j] = \min\{ c^{k-1}[i, j],\ c^{k-1}[i, k] + c^{k-1}[k, j] \}$

Repeated Single-Source Shortest Path Algorithms

- Nonnegative edge costs: n x Dijkstra:

  n x $O(m + n \log n) = O(mn + n^2 \log n)$.

- Negative edge costs: n x Bellman-Ford:

  n x $O(mn) = O(mn^2)$.

  (this can be improved to $O(mn + n^2 \log n)$ by using Johnson's algorithm: run Bellman-Ford once, then "reweight" the edges of the graph with nonnegative costs, and run Dijkstra n times…).

Johnson's Algorithm

- Use Bellman-Ford to find the shortest path cost c[i] from an arbitrary source s to every other node i.
  - Sometimes introduce dummy source s connected to all other edges with zero-cost edges (to ensure every node reachable from s).
- These costs c[i] satisfy the triangle inequality:

  $c[i] + cost(i, j) \geq c[j]$ for every edge (i, j).
- Define the **reduced cost** of (i, j) as:

  $cost'(i, j) = cost(i, j) + c[i] - c[j] \geq 0$.
- Now find shortest paths from every other source node using Dijkstra's algorithm!
- Total running time: $O(mn + n^2 \log n)$.